Embedding Flaris in a C application

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.6.0 **Generated** 2026-10-27 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Flaris ships as a library as well as a program. Link `libflaris` into your application and Flaris becomes its scripting layer: start a host, load scripts into it, and call their functions from C.

This is the counterpart to the FFI guide. FFI is *a script reaching a shared library you did not compile against*; this is *your application and a script calling each other directly*, in the same process, with nothing copied between them.

- 1. What you download
- 2. Hello, host
- 3. Shipping scripts with your application
- 4. Lifecycle
- 5. Configuration
- 6. Loading code
- 7. Calling into Flaris
- 8. Values and ownership
- 9. Native modules: your C functions, called from script
- 10. Restricting what a script can reach
- 11. Signals from the script
- 12. Errors and diagnostics
- 13. Threads: one host per thread
- 14. Embedding from C#
- 15. Embedding from JavaScript
- 16. Limits and what is not supported
- 17. API reference

---

1. What you download

Get the library for your platform from the downloads page. Each archive unpacks to the same shape:

```
flaris-lib/
├── include/
│   └── flaris.h       # the embedding API — the only header you include
└── lib/               # the static and shared library
```

One header, and it is self-contained: it declares the whole API in terms of C types, so nothing about the VM's internals leaks into your build. Values cross the boundary as `FlarisValue`, an opaque one-word handle.

Platform	Static	Shared
macOS ARM64	`libflaris.a`	`libflaris.dylib`
Linux x86_64 / ARM64	`libflaris.a`	`libflaris.so`
Windows x64	`libflaris.a`	`flaris.dll` + `libflaris.dll.a`

This is the same VM the `flarisvm` command runs — compiler, runtime, JIT, standard builtins and debugger — without the command-line front end. Nothing a script does, and nothing a bad input file contains, makes it call `exit()`; your application stays in control. Section 12 states the guarantee exactly.

Include path and link flags. Point your compiler at `include/`, link the library, and add the system libraries the VM needs:

```
# macOS
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
   -lm -framework Security -framework CoreFoundation -lobjc

# Linux
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
   -lm -ldl -lpthread

# Windows (MSYS2 clang64)
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
   -lws2_32 -lbcrypt -liphlpapi -lsecur32 -lcrypt32
```

Linking the shared library instead. The shared library exports exactly what this document describes - the `Flaris*` functions listed in section 17, plus `flarisHostOptionsDefaults` - and nothing else; the VM's internals are hidden, so no future release can break you by moving one. On Windows, add `-DFLARIS_DLL` when you build against `flaris.dll`, so the declarations become `dllimport`:

```
cc host.c -DFLARIS_DLL flaris-lib/lib/libflaris.dll.a -Iflaris-lib/include -o host
```

macOS and Linux need no define, but they do need an rpath, or the executable will not find the library beside it at launch:

```
# macOS
cc host.c flaris-lib/lib/libflaris.dylib -Iflaris-lib/include \
   -Wl,-rpath,@executable_path -o host

# Linux
cc host.c flaris-lib/lib/libflaris.so -Iflaris-lib/include \
   -Wl,-rpath,'$ORIGIN' -o host
```

---

2. Hello, host

Complete and compilable — nothing is elided.

```c
#include "flaris.h"
#include <stdio.h>

static const char *SCRIPT =
    "fn Greet(who: string): string {\n"
    "    return \"hello, \" + who;\n"
    "}\n";

int main(void)
{
    FlarisHost *host;
    if (FlarisHostCreate(NULL, &host) != FLARIS_OK)
        return 1;

    FlarisFunction *greet = NULL;
    if (FlarisHostLoadScript(host, SCRIPT, "greet.fls") == FLARIS_OK)
        greet = FlarisHostGetFunction(host, "Greet");
    if (!greet) {
        FlarisHostDestroy(host);
        return 1;
    }

    FlarisValue args[1] = { FlarisString("world") };
    FlarisValue result;
    char err[256];

    int rc = FlarisFunctionCall(greet, args, 1, &result, err, sizeof err);
    if (rc == FLARIS_OK) {
        size_t len = 0;
        const char *text = FlarisAsString(result, &len);
        printf("%.*s\n", (int)len, text);
        FlarisReleaseValue(result);
    } else {
        printf("error: %s\n", err);
    }

    FlarisReleaseValue(args[0]);   /* the call borrows; you still own it */
    FlarisHostDestroy(host);       /* every value and handle dies here */
    return 0;
}
```

```
$ cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
     -lm -framework Security -framework CoreFoundation -lobjc
$ ./host
hello, world
```

A few things in that program are worth naming now, because everything else builds on them.

A **host** is one VM, and it belongs to the thread that created it. Every script you load into it lands in its one global scope, and `FlarisHostDestroy` takes all of it down at once. Section 4 is about its life.

A `FlarisFunction` is a handle to a function the script defined. Resolve it once by name and call it as often as you like; the host owns it, so there is nothing to release.

`FlarisValue` is an **opaque handle** to a value living inside the VM. It is a single word; you never dereference it, and every read goes through a function like `FlarisAsString`. Nothing is copied when one crosses the boundary.

`FlarisFunctionCall` **borrows** its arguments — you built them, you release them. The **result is yours**, and `FlarisReleaseValue` is how you drop it.

Every entry point returns a status code rather than aborting. A script that throws, a file that will not compile, a corrupt `.flx` and a module whose pinned hash does not match are all ordinary return values, and a function that does not exist is a `NULL` handle.

---

3. Shipping scripts with your application

You can hand Flaris either source (`.fls`) or compiled bytecode (`.flx`). For anything you ship, prefer bytecode:

- it does not expose your script source,
- it skips compilation at startup,
- it is validated on load, so a corrupt or mismatched file is refused rather than half-run.

Compile with the `flarisvm` toolchain as part of your build:

```
flarisvm --compile game.fls game.flx
```

Then load it at runtime:

```
FlarisHostLoadProgram(host, "game.flx");
```

A script you load this way is a **library of functions**, not a program. The host needs neither a `Main` nor an `export` list, and calls neither. `--compile` does insist on one of the two, so give a script you compile ahead of time an `export` list:

```
fn Greet(who: string): string { return "hello, " + who; }
fn Update(dt: float): int    { /* ... */ return 0; }

export { Greet, Update };
```

Everything the script declares is callable by name afterwards, whether or not it appears in the `export` list — the list is there to satisfy the compiler and to document intent. A `Main`, if the script has one, is loaded like any other function and runs only if you call it.

> Bytecode is portable across platforms and machines: a `.flx` built on one target runs on any other.

---

4. Lifecycle

```
FlarisHostCreate(&opt, &host)                    ← one per thread at a time
      │
      ├─ FlarisRegisterModule(host, "Device", ...)  ← optional; or opt.modules
      ├─ FlarisHostLoadProgram(host, "game.flx")    ← or LoadSource / LoadScript
      ├─ fn = FlarisHostGetFunction(host, "Update") ← resolve once
      ├─ FlarisFunctionCall(fn, ...)                ← as often as you like
      │
FlarisHostDestroy(host)
```

A host belongs to its thread. The thread that calls `FlarisHostCreate` owns the host, and has at most one at a time: a second create on the same thread returns `FLARIS_ERR_BUSY` until the first is destroyed. Any number of other threads may each have their own (section 13). A call through the host, or through a function it handed out, from any other thread returns `FLARIS_ERR_WRONG_THREAD`.

`FlarisHostDestroy` ends everything the host holds: its scripts and their globals, its fibers, timers and I/O, its host modules, and every `FlarisValue` and `FlarisFunction` it handed out — including values you retained. Copy what you want to keep into your own C data first. The thread is free for a new host afterwards.

It returns `FLARIS_ERR_BUSY`, and destroys nothing, when it is called from inside one of the host's own calls — a native module function cannot destroy the VM it is running in. Return from the call and destroy it then.

What a host is given ends with it. Its capability grants, its limits, its denied modules, its host modules, its output handler (section 12) and its notify handler (section 11) all go with it, and the next host on that thread starts from its own options and the defaults, never from what the previous one was allowed. One setting is not part of a host: `FlarisSetArgs`, which is one argv for the whole process.

Starting over is cheap. Creating a host, loading a script, calling one function and destroying the host again takes tens of microseconds, so a host per job, per request or per test is a reasonable shape. To change a single function without starting over, load a script that redefines it (section 6).

---

5. Configuration

Pass `NULL` to `FlarisHostCreate` for defaults, or fill in a `FlarisHostOptions`:

```
FlarisHostOptions opt = flarisHostOptionsDefaults;
opt.installSignals = FLARIS_OFF;         /* keep your own signal handlers */
opt.startIoPool    = FLARIS_OFF;         /* stay single-threaded          */
opt.grantCaps      = FLARIS_CAP_UNSAFE;  /* the script may use Ffi/Memory */
opt.maxFibers      = 64;                 /* power of two                  */
opt.stackSize      = 8192;
opt.libsPath       = "/opt/myapp/scripts";
```

```
FlarisHost *host;
if (FlarisHostCreate(&opt, &host) != FLARIS_OK) {
    /* an option was rejected — nothing was started */
}
```

The options are read once, at create. They hold for the host's whole life and end with it.

Switches

Every switch is tri-state: `FLARIS_DEFAULT` (which is `0`, so a zeroed struct is a valid starting point), `FLARIS_ON`, or `FLARIS_OFF`. Anything else is rejected rather than read as a default.

Switch	Default	What it does
`installSignals`	on	Installs the VM's handlers for `SIGINT`, `SIGTERM`, `SIGHUP`, `SIGUSR1` and `SIGUSR2` (and ignores `SIGPIPE`), so a script can receive them through `VM.OnSignal`. A signal no script handler has claimed ends the process at once with exit code 128 + signal number, nothing torn down. Off leaves your handlers alone. The VM installs no `SIGSEGV` handler either way — a fault inside it reaches *you*.
`startIoPool`	on	Starts the I/O worker threads. Off keeps your process single-threaded; I/O still completes, inline on the host's thread.
`jit`	on	Native code generation.
`optimizations`	on	Constant folding and peephole passes.
`debugInfo`	off	Keeps source line numbers, so stack traces report real lines instead of line 0. Function and file names are kept either way.
`signatureChecks`	on	Verifies a `.flx` fingerprint when one is pinned.
`requireSigned`	off	Refuses `.flx` not signed by a trusted key.
`verbose`	off	Emits the VM's own progress chatter at `FLARIS_LOG_INFO`.
`colors`	auto	ANSI colour in diagnostics: on when stderr is a terminal and `NO_COLOR` is unset. Irrelevant once you set an output handler.

Capabilities

What a script may *do* is a bitmask rather than a switch, set with `opt.grantCaps` and `opt.denyCaps` (deny is applied after grant, so a bit in both is withheld):

Capability	Default	What it allows
`FLARIS_CAP_UNSAFE`	off	`Ffi.*`, raw `Memory` access and `Buffer` addresses. Implies `FLARIS_CAP_FFI`.
`FLARIS_CAP_FFI`	off	`Ffi.*` alone, without the rest of `UNSAFE`. Calling a native function object needs it too.
`FLARIS_CAP_IMPORT_LOCAL`	**on**	`import` may resolve from disk and `libsPath`.
`FLARIS_CAP_IMPORT_REMOTE`	off	`import` may fetch over `https://`.
`FLARIS_CAP_IMPORT_INSECURE`	off	...and over plain `http://` too. Needs `REMOTE` as well.

The host sets them once. A **script** can never widen them: a fiber it spawns is born with its creator's capabilities and may only narrow from there.

> `FLARIS_CAP_UNSAFE` **is a trust decision.** It lets a script load native code into your process via `Ffi` and read or write raw addresses via `Memory`. Leave it off for scripts you do not control.

> **The import capabilities are the other one.** A script that may fetch and run code from a URL is only as trustworthy as that URL. The default is local imports only; the `flarisvm` command grants `IMPORT_REMOTE` itself, because it runs what the user chose to run.

Limits

Limit	Default	Range
`stackSize`	1024	64–65535 value-stack slots
`maxSlabs`	1024	object-pool ceiling
`maxFibers`	256	power of two, 2–1024
`maxFrames`	64	8–1024 call frames per fiber
`ioThreads`	0	0 scales to the CPU count; ≤ 16
`libsPath`	none	Where `import` looks, `;`-separated. **Borrowed** — must outlive the host.

Every limit reads `0` as "keep the default", so you set only what you care about. An out-of-range value makes `FlarisHostCreate` **fail** with `FLARIS_ERR_INIT` rather than being quietly clamped, and nothing is started when it does.

`maxSlabs` is the host's memory ceiling: a script that allocates past it gets `Exception.OutOfMemory`, which it may catch and which otherwise reaches you as an ordinary raise (section 12). `maxFrames` bounds recursion: a call frame carries the locals array, so it costs about a kilobyte. A fiber makes its frames as it first calls that deep and keeps them for its next calls, so one that stays a few calls deep holds a few kilobytes, and only a fiber that reaches the limit holds all 64. Exceeding it raises a catchable stack exception.

Host modules and the rest

`modules` and `moduleCount` publish your C functions to scripts; section 9 covers them. `deniedModules` withholds built-in modules; section 10 covers that. `output` and `outputUserData` take everything the host writes — script output and diagnostics; section 12 covers them.

---

6. Loading code

Code is loaded **into a host**:

```
FlarisHostLoadProgram(host, "game.flx");        /* bytecode - the shipping path  */
FlarisHostLoadSource(host, "game.fls");         /* source file, compiled on load */
FlarisHostLoadScript(host, text, "rules.fls");  /* source from memory            */
```

Each one runs the script's **top level** — global initialisers and function definitions — and stops. None of them calls `Main`, and none of them requires the script to have one. The `name` that `FlarisHostLoadScript` takes is what diagnostics call the script; `NULL` means `"<host>"`.

Return	Meaning
`FLARIS_OK`	the top level ran; its functions are callable
`FLARIS_ERR_COMPILE`	the script did not compile, the file could not be read, or the `.flx` was refused
`FLARIS_ERR_RAISED`	the top level threw
`FLARIS_ERR_SUSPENDED`	the top level waited (see below)

Return	Meaning
`FLARIS_ERR_WRONG_THREAD`	the host belongs to another thread

A runtime-only build of the library has no compiler, so `FlarisHostLoadProgram` is the only one of the three it can use; the source forms return `FLARIS_ERR_COMPILE`.

One global scope

Every load lands in the host's **one global scope**, so what a script defines stays defined when the next one loads, and you may load as many as you like. Each script still compiles on its own: it cannot call a function or read a global that another load defined — only the host sees all of them. A later load that defines a name an earlier one already used **replaces** it — the VM reports the rebinding as a warning — and that is how you swap one function without starting the host over:

```
FlarisHostLoadScript(host, "fn Bonus(score: int): int { return score + 10; }\n", "rules.fls");
FlarisHostLoadScript(host, "fn Bonus(score: int): int { return score + 25; }\n", "patch.fls");
/* Bonus now adds 25 - resolve it again to call the new one */
```

The replacement is for the host. A function the first script defined that calls `Bonus` keeps calling the version it was loaded with, so patch a function together with its callers.

The flip side is that two scripts which both declare `config`, or both define `Init`, overwrite each other. Load into one host only scripts written to share a namespace. Scripts that must not see each other belong in separate hosts, which means separate threads (section 13).

A top level runs to the end, but may not wait

However long a global initialiser takes, it finishes: the scheduling quantum does not truncate it. What it may **not** do is wait. A top level that calls `Fiber.Sleep`, awaits, starts a fiber or parks on I/O returns `FLARIS_ERR_SUSPENDED` and is abandoned where it stopped. Whatever it had already defined stays defined; the declarations after that point never land.

Do the waiting inside a function — and read section 7 on what a call may do.

---

7. Calling into Flaris

Resolve a function by name once, then call it through the handle:

```
FlarisFunction *FlarisHostGetFunction(FlarisHost *host, const char *name);

int FlarisFunctionCall(FlarisFunction *fn, const FlarisValue *args, int argc,
                       FlarisValue *outResult, char *errBuf, size_t errSize);
```

`FlarisFunctionCall` **borrows** its arguments: you built them, you release them, and the same values may be passed to several calls. It hands back an **owned** result you release with `FlarisReleaseValue`, or pass `NULL` to have it released for you.

```
FlarisFunction *damage = FlarisHostGetFunction(host, "Damage");

FlarisValue args[2] = { FlarisString("player"), FlarisInt(3) };
FlarisValue result;
char err[256];

int rc = FlarisFunctionCall(damage, args, 2, &result, err, sizeof err);
if (rc == FLARIS_OK) {
    printf("hp now %lld\n", (long long)FlarisAsInt(result));
    FlarisReleaseValue(result);
}
FlarisReleaseValue(args[0]);
FlarisReleaseValue(args[1]);   /* a small int allocates nothing; releasing it
                                  is still correct and still free */
```

Return	Meaning
`FLARIS_OK`	`result` holds the return value
`FLARIS_ERR_RAISED`	the script threw; `err` holds `"code: message"`
`FLARIS_ERR_SUSPENDED`	a plain function waited, and was abandoned (see below)
`FLARIS_ERR_ARGS`	`fn` is `NULL`, `args` is `NULL` with `argc` > 0, or `argc` is negative or above `FLARIS_MAX_CALL_ARGS` (16)
`FLARIS_ERR_WRONG_THREAD`	`fn` belongs to a host on another thread (section 13)

On anything but `FLARIS_OK`, `result` holds nil, so releasing it is harmless. Passing the wrong number of arguments for the function itself — too few, say — raises in the script like any other call mistake and comes back as `FLARIS_ERR_RAISED`. A longer list than 16 is refused rather than truncated.

The function handle

`FlarisHostGetFunction` returns `NULL` when nothing callable has that name: a name no script defined, a global holding something that is not a function, a class, or a built-in. It finds what your scripts bound at their top level — functions, and a lambda stored in a global. That makes it the test for optional hooks:

```
FlarisFunction *onJoin = FlarisHostGetFunction(host, "OnPlayerJoin");

/* later, as often as it happens */
if (onJoin) {
    FlarisValue a = FlarisString(playerName);
    FlarisFunctionCall(onJoin, &a, 1, NULL, err, sizeof err);
    FlarisReleaseValue(a);
}
```

A handle is **owned by the host** and valid until `FlarisHostDestroy`; there is nothing to release. Resolving the same function again returns the same handle, so resolving by name in a loop costs a lookup but accumulates nothing — still, resolve once and keep it, since a call through a handle skips the lookup entirely.

A handle **keeps the function it resolved**. When a later load redefines the name (section 6), the handle you already hold still calls the old function; resolve the name again to get the new one.

`FlarisIsCallable` tells you that a value is a function — a bound method on an instance, say — but the host calls only what `FlarisHostGetFunction` resolves. To call a method, have the script define a function that calls it.

Calls may nest

A native module function (section 9) reached from a call may itself call `FlarisFunctionCall` on the same host. The nested call runs above the outer one's frames and leaves them exactly as it found them, and the outer call carries on when it returns.

Calls that wait

A call runs on your thread and returns when the function does. When the function is an `fn async` — the only kind that may use `await` — the call waits for it. While it waits, the host runs its own loop on your thread: timers fire, asynchronous I/O completes, other fibers get their turn, and the thread sleeps whenever nothing is ready. When the function returns, so does the call, with its value; an exception that escapes it comes back as `FLARIS_ERR_RAISED`.

```
FlarisHostLoadScript(host,
    "fn async Report(id: int): string {\n"
    "  let row = await Db.Fetch(id);    // waits on I/O\n"
    "  return Format(row);\n"
    "}\n", "report.fls");
FlarisFunction *report = FlarisHostGetFunction(host, "Report");
FlarisFunctionCall(report, &id, 1, &result, err, sizeof err);  /* returns when Report does */
```

The call blocks your thread for as long as the script waits — 150 ms if it awaits a 150 ms fetch. On a server that is the natural shape: give each host its own thread, and a request that waits holds only that host.

Three things a call does not do:

- **Wait inside a plain function.** A function that is not `async` but still waits — `Fiber.Sleep`, `Fiber.Await`, a stream read that parks until data arrives, `Fiber.Run` starting a fiber — returns `FLARIS_ERR_SUSPENDED` and is abandoned where it stopped; its half-finished frames are unwound and the host stays usable. Make the function `fn async` instead.
- **Wait from inside a host function.** A native module function (section 9) that calls an `fn async` function gets `FLARIS_ERR_SUSPENDED`: the call it is nested in is still on the stack below it. Only the outermost call waits.
- **Run anything between calls.** Work a call leaves behind — a timer it armed, a fiber it started and did not await — moves on only while some later call is waiting or a fiber is stepped, and is torn down with the host.

A call waits as long as the script does: one that awaits something that never comes holds your thread until it does. To bound the wait, start the call and step it instead.

Starting a call and stepping it

```
FlarisValue FlarisFunctionCallAsync(FlarisFunction *fn, const FlarisValue *args, int argc);
```

```
int FlarisFiberStep(FlarisHost *host, FlarisValue fiber, int maxWaitMs,
                    FlarisValue *outResult, char *errBuf, size_t errSize);

int FlarisFiberCancel(FlarisHost *host, FlarisValue fiber);
```

`FlarisFunctionCallAsync` starts an `fn async` function and returns at once with its **fiber** — the same fiber a script gets when it calls an async function without awaiting it. Nothing has run yet. Each `FlarisFiberStep` moves the host on until that fiber finishes or `maxWaitMs` has passed, so the clock is yours:

```
FlarisValue fiber = FlarisFunctionCallAsync(report, &id, 1), result;
double deadline = NowMs() + 2000;
int rc;
while ((rc = FlarisFiberStep(host, fiber, 10, &result, err, sizeof err)) == FLARIS_PENDING) {
    if (NowMs() > deadline) {      /* your clock, your cancel button, your frame budget */
        FlarisFiberCancel(host, fiber);
        break;
    }
    DoOtherWork();
}
if (rc == FLARIS_OK) {
    Use(result);
    FlarisReleaseValue(result);
}
FlarisReleaseValue(fiber);
```

`maxWaitMs`	A step
`0`	runs what is ready once and returns, never sleeping — one step per frame in a game loop
above 0	runs, and sleeps while nothing is ready, returning by then at the latest
`-1`	returns only once the fiber has finished — what `FlarisFunctionCall` does

A step moves the **whole host**, not only that fiber: the fiber may be waiting on a timer, on I/O or on another fiber, and those need their turn. With several fibers in flight, stepping any one of them moves them all — step one with a wait, and look at the others with `0`.

Return	Meaning
`FLARIS_PENDING`	it has not finished; step again
`FLARIS_OK`	it returned; `result` holds the value, owned
`FLARIS_ERR_RAISED`	an exception escaped it; `err` holds `"code: message"`
`FLARIS_ERR_SUSPENDED`	it was cancelled, or nothing left could ever wake it
`FLARIS_ERR_ARGS`	`fiber` is nil, or not a fiber `FlarisFunctionCallAsync` returned
`FLARIS_ERR_BUSY`	called from inside a native module function, while the host is already running
`FLARIS_ERR_WRONG_THREAD`	the host belongs to another thread (section 13)

Once the fiber has finished, every step answers the same: the value or the error stays with the fiber until you release it.

`FlarisFunctionCallAsync` **borrows** its arguments and returns an **owned** fiber. It returns nil only when the call itself is wrong — `fn` is `NULL` or not `fn async`, there are too many arguments, or the host belongs to another thread — and stepping nil answers `FLARIS_ERR_ARGS`, so the loop above needs no separate check. A

start the script refuses — an argument of the wrong type, or no fiber free under `maxFibers` — still returns a fiber, and its first step reports why.

`FlarisFiberCancel` stops the fiber where it is: it never runs again, and its `finally` blocks do not run. A fiber that has already finished keeps its value. Releasing a fiber that has not finished does **not** stop it: it runs on whenever the host is stepped or waits in a call, and its result is dropped. Fibers still pending when the host is destroyed are torn down with it.

A step returns on time even while the fiber computes without waiting: the host takes turns in slices, and checks the clock between them. A function compiled to native code (section 5) is the exception — it runs to its end before the step can return, so a long native loop holds the step for as long as it runs.

---

8. Values and ownership

A `FlarisValue` is a one-word opaque handle to a value inside the VM. Do not dereference it, do not store it in a struct expecting to inspect it, and do not assume anything about its bit pattern. Everything you can do with one is a function call.

Making values

```
FlarisValue v;

v = FlarisNil();
v = FlarisBool(1);
v = FlarisInt(42);
v = FlarisFloat(3.5);
v = FlarisString("text");            /* NUL-terminated */
v = FlarisStringN(buf, len);         /* explicit length; may contain NULs */
v = FlarisArrayNew(16);              /* capacity hint, starts empty */
v = FlarisObjectNew();               /* string-keyed */
```

Each of these returns a value **you own**. A value is made in the host of the calling thread, so make values only while that host is alive.

Reading values

Test first, then read. A read of the wrong type gives `0` or `NULL` rather than faulting, so a missing check misbehaves visibly instead of crashing.

```
if (FlarisIsInt(v))    n = FlarisAsInt(v);     /* int64_t */
if (FlarisIsFloat(v)) d = FlarisAsFloat(v);   /* double  */
if (FlarisIsBool(v))   b = FlarisAsBool(v);    /* int     */

if (FlarisIsString(v)) {
    size_t len = 0;
    const char *p = FlarisAsString(v, &len);   /* borrowed, not a copy */
```

```
    fwrite(p, 1, len, stdout);
}
```

`FlarisAsString` hands back a pointer into the VM's own storage. It is valid while the value is, and it is not yours to free. Copy it if you need to keep it. It returns `NULL` for anything that is not a string — it will not coerce a number into text for you.

Arrays and objects

```
/* build */
FlarisValue a = FlarisArrayNew(3);
FlarisArrayAppend(a, FlarisInt(1));      /* append CONSUMES the value */
FlarisArrayAppend(a, FlarisInt(2));

FlarisValue o = FlarisObjectNew();
FlarisObjectSet(o, "name", FlarisString("sensor"));   /* set CONSUMES too */
FlarisObjectSet(o, "id", FlarisInt(7));

/* read */
int n = FlarisArrayLen(a);
for (int i = 0; i < n; i++) {
    FlarisValue e = FlarisArrayGet(a, i);   /* OWNED - release it */
    Consume(FlarisAsInt(e));
    FlarisReleaseValue(e);
}

FlarisValue name = FlarisObjectGet(o, "name");   /* borrowed */
```

For numeric arrays, skip the handle entirely — these allocate nothing and need no release, which matters when you are walking thousands of elements:

```
int64_t x = FlarisArrayGetInt(a, i);
double  y = FlarisArrayGetFloat(a, i);
```

And for bulk binary data, a `Block` hands you its bytes directly, so there is no per-element call at all:

```
size_t len = 0;
unsigned char *bytes = FlarisBlockData(v, &len);
```

`FlarisObjectNext` walks an object's properties in insertion order, key and value both borrowed: start the cursor at `0` and call until it returns `0`, and do not add or remove properties meanwhile. A **class instance** is not an object by `FlarisIsObject` — `FlarisIsInstance` is its test and `FlarisClassName` names its class — but the object accessors read and write its fields in place, exactly as they do an object's properties.

The ownership rules

There are three, and they are the same rules in both directions.

1. What you make, you own. Every constructor above returns a value with one reference belonging to you. Release it with `FlarisReleaseValue` when you are done.

2. Some calls take it off your hands. `FlarisArrayAppend` and `FlarisObjectSet` **consume** the value you pass — the container takes your reference. Do not release it afterwards. Returning a value from a native module function (section 9) consumes it the same way: the VM takes it.

3. Borrowed values are not yours. Arguments handed to your native module function are **borrowed** — valid for that call only. If you need to keep one past the call, take your own reference:

```
FlarisValue kept = FlarisRetain(args[0]);
/* ... later ... */
FlarisReleaseValue(kept);
```

`FlarisObjectGet` also borrows. `FlarisArrayGet` does **not** — it returns an owned value, because a `[float]` array stores raw doubles and has no element object to lend you.

Scalars — nil, bool, small ints, chars — allocate nothing, so retaining and releasing them is free. You still write the calls; they simply cost nothing.

Everything dies with its host

Every `FlarisValue` and every `FlarisFunction` belongs to the host that made it. `FlarisHostDestroy` ends them all — a value you retained included — so release what you hold before you destroy the host, and copy anything you want to keep into plain C data first. A value from one host means nothing to another (section 13).

---

9. Native modules: your C functions, called from script

Publish C functions as a module and scripts call them by name, exactly as they call a built-in namespace. Nothing is marshalled at the boundary — your function receives the same handles the script is holding.

A complete example

```
#include "flaris.h"
#include <stdio.h>

/* ── stand-ins for your hardware ──────────────────────────────── */

static double ReadSensor(int pin) { return pin * 0.25; }

static void WriteChannel(int channel, double value)
{
    printf("channel %d <- %.2f\n", channel, value);
}

/* ── the functions scripts will call ──────────────────────────── */
```

```
static double gLastOutput;

/* Device.Read(pin) -> float */
static FlarisValue DeviceRead(FlarisValue *args, int argc)
{
    (void)argc;
    return FlarisFloat(ReadSensor((int)FlarisAsInt(args[0])));
}

/* Device.SetOutput(channel, value) -> nil */
static FlarisValue DeviceSetOutput(FlarisValue *args, int argc)
{
    (void)argc;
    gLastOutput = FlarisAsFloat(args[1]);
    WriteChannel((int)FlarisAsInt(args[0]), gLastOutput);
    return FlarisNil();          /* a "void" function returns nil */
}

/* Device.Status() -> object */
static FlarisValue DeviceStatus(FlarisValue *args, int argc)
{
    (void)args; (void)argc;
    FlarisValue o = FlarisObjectNew();
    FlarisObjectSet(o, "output", FlarisFloat(gLastOutput));
    FlarisObjectSet(o, "online", FlarisBool(1));
    return o;                    /* the VM takes this reference */
}

/* ── the table ──────────────────────────────────────────────────────────── */
/*  name        function          returns          argument types                req opt flags */
static const FlarisNative DEVICE[] = {
    {"Read",      DeviceRead,      FLARIS_T_FLOAT,  {FLARIS_T_INT},                1, 0,
FLARIS_FN_JIT_SAFE},
    {"SetOutput", DeviceSetOutput, FLARIS_T_NIL,    {FLARIS_T_INT, FLARIS_T_FLOAT}, 2, 0, 0},
    {"Status",    DeviceStatus,    FLARIS_T_OBJECT, {0},                            0, 0, 0},
};

/* ── wiring ─────────────────────────────────────────────────────────────── */

static const char *SCRIPT =
    "fn Sample(pin: int): float {\n"
    "    let v = Device.Read(pin);\n"
    "    Device.SetOutput(1, v * 2.0);\n"
    "    return v;\n"
    "}\n";

int main(void)
```

```
{
    /* In the options, so the module exists before any script is compiled. */
    FlarisModule device = { "Device", DEVICE, 3 };

    FlarisHostOptions opt = flarisHostOptionsDefaults;
    opt.modules     = &device;
    opt.moduleCount = 1;

    FlarisHost *host;
    if (FlarisHostCreate(&opt, &host) != FLARIS_OK)
        return 1;

    FlarisFunction *sample = NULL;
    if (FlarisHostLoadScript(host, SCRIPT, "control.fls") == FLARIS_OK)
        sample = FlarisHostGetFunction(host, "Sample");
    if (!sample) {
        FlarisHostDestroy(host);
        return 1;
    }

    FlarisValue pin = FlarisInt(4);
    FlarisValue out;
    char err[256];

    if (FlarisFunctionCall(sample, &pin, 1, &out, err, sizeof err) == FLARIS_OK) {
        printf("read %.2f\n", FlarisAsFloat(out));
        FlarisReleaseValue(out);
    } else {
        printf("error: %s\n", err);
    }

    FlarisReleaseValue(pin);
    FlarisHostDestroy(host);
    return 0;
}
```

```
$ ./host
channel 1 <- 2.00
read 1.00
```

The script writes `Device.Read(pin)` the way it writes `Math.Abs(x)`. Because the compiler knows the signature you registered, passing a string where an int is declared is a **compile error**, not a runtime surprise.

Two ways to register

In the options, at create — `opt.modules` points at an array of `FlarisModule { name, fns, count }` and `opt.moduleCount` says how many. This is the usual shape: the modules exist before anything is loaded, for

the host's whole life.

After create, for a host that learns its modules later — plugins discovered at runtime, say:

```
int rc = FlarisRegisterModule(host, "Device", DEVICE, 3);
```

Either way the VM **copies** the table and its names, so both may be temporaries. And either way only scripts loaded **afterwards** can call the module: a script is compiled against the modules registered at the moment it loads, so a script that calls `Device.*` and is loaded first fails to compile.

A registration that fails — `FLARIS_ERR_REGISTERED` for a name already taken, `FLARIS_ERR_COLLISION` for two names sharing a hash, `FLARIS_ERR_ARGS` for an empty or malformed table — registers nothing. In the options, the same codes come back from `FlarisHostCreate`, and no host is started.

Declaring the signature

`returnType` and each entry in `argTypes` is one of:

```
FLARIS_T_NIL     FLARIS_T_BOOL    FLARIS_T_INT     FLARIS_T_FLOAT
FLARIS_T_CHAR    FLARIS_T_STRING  FLARIS_T_ARRAY   FLARIS_T_OBJECT
FLARIS_T_BLOCK   FLARIS_T_ANY
```

They are bit flags, so `FLARIS_T_INT | FLARIS_T_FLOAT` accepts either. `0` in a slot means the same as `FLARIS_T_ANY`.

`required` is how many arguments must be supplied and `optional` how many more may be. A function declaring `2, 1` accepts two or three; check `argc` yourself to see which you got. The maximum is six (`FLARIS_MAX_HOST_ARGS`).

Ownership inside a native function

Two rules, both from section 8:

- **`args` are borrowed.** Valid for the duration of the call. To keep one, take a reference with `FlarisRetain` and release it later.
- **Your return value is consumed.** The VM takes the reference. Return `FlarisNil()` if you have nothing to give back — never a value you also released.

Making the calls fast

By default, a script function containing a call into your module is **not JIT-compiled at all** — one call the compiler cannot make discards the native code for the whole enclosing function. `FLARIS_FN_JIT_SAFE` lifts that: native code may call your function directly, with its arguments borrowed, instead of going through the interpreter. In a hot loop that is the difference between an interpreted loop and a native one.

It is a promise the VM cannot verify, so set it only when your function:

- does not yield, await or suspend;
- does not call back into Flaris;
- does not keep a borrowed argument past the call without `FlarisRetain`;

- does not assume an interpreter frame exists.

Raising an error and having side effects are both fine. `FLARIS_FN_PURE` is a stronger claim — that the result depends only on the arguments and the call has no observable effect — which additionally lets the optimiser reorder, hoist or drop the call, and implies the JIT-safe promise. Do not set it on anything reading a device, a clock, a counter or any shared state. A function that breaks either promise produces wrong code, not a diagnostic.

Flag the functions your hot paths call. Flagging one function in a module whose others are unflagged still leaves any script function calling the others interpreted.

Rules and limits

- **Register before you load.** The compiler types these calls from the registry and the loader binds them; a module registered afterwards does not reach a script that is already loaded.
- **A module belongs to its host.** It is registered for that host alone and goes when the host is destroyed; a host on another thread registers its own.
- **Names are matched by hash.** Two names that collide in 32 bits cannot both be registered — you get `FLARIS_ERR_COLLISION` and rename one. A name that does not resolve when a script loads rejects that script rather than failing later at the call.
- **A module name cannot shadow a built-in** (`Math`, `Json`, `File`, …) or a module already registered — that returns `FLARIS_ERR_REGISTERED`.
- **Compiled scripts become yours.** A `.flx` containing calls into your module runs only in a host that registers those exact names. That is the intended shape: these are your application's scripts, compiled when your host loads them, not artifacts meant to run under a plain `flarisvm`.
- **No `unsafe` needed.** Unlike the FFI plugin path, a native module works with the default configuration.

---

10. Restricting what a script can reach

By default a script can use every built-in module: the filesystem, sockets, processes, everything. A host withholds whole modules with a bitmask, one bit per module:

```
FlarisHostOptions opt = flarisHostOptionsDefaults;
opt.deniedModules = FLARIS_MOD_FILE | FLARIS_MOD_DIRECTORY |
                    FLARIS_MOD_FILEWATCH |
                    FLARIS_MOD_NET  | FLARIS_MOD_OS |
                    FLARIS_MOD_FFI  | FLARIS_MOD_BUFFER | FLARIS_MOD_MEMORY;

FlarisHost *host;
FlarisHostCreate(&opt, &host);
```

A call into a denied module raises `Exception.ModuleDenied` (code 25), which the script can catch and your `FlarisFunctionCall` sees as `FLARIS_ERR_RAISED`. Nothing else about the script changes, and an allowed module costs nothing — the check is a bit test on an operand the call already carries.

JIT-compiled code is held to the same mask. A compiled function knows which built-in modules its native code can reach, counting every compiled function it calls; in a host that denies one of them it runs interpreted, so

the call raises exactly where it would with the JIT off. That covers callbacks a built-in runs for the script (`Array.Map`, `Memory.Process`, ...) too.

The constants are `FLARIS_MOD_` plus the module name in upper case: `FLARIS_MOD_FILE`, `FLARIS_MOD_NET`, `FLARIS_MOD_OS`, `FLARIS_MOD_JSON` and so on, one for each of the 32 built-in modules. They are positional, so recompile against the header you ship with.

TLS is not separately gated. A client speaking `https://` is doing what `Net` and `Stream` are for; if a script should not reach the network, deny those modules. What does need its own gate is a script's ability to **load more code**, which is what the `FLARIS_CAP_IMPORT_*` capabilities in section 5 are for.

What this is, and what it is not

It is **refusal, not confinement**. You can remove the filesystem; you cannot grant a subdirectory. There is no path allow-list, no host-supplied resolver and no per-call hook.

The unit is the **whole module**. Denying `FLARIS_MOD_OS` removes `Os.Execute` and `Os.Spawn`, and also `Os.Args` and `Os.Arch`, which only report the environment. It errs towards denying more.

And it bounds **capability, not consumption**. `maxFrames` bounds how deep a script can recurse and `maxSlabs` how much it can allocate (section 5), but nothing bounds how long a call runs: a script with every module denied can still loop forever, the VM has no timeout of its own, and there is currently no way to stop a running call from another thread. If you are running code you do not trust at all, run its host in a process you can kill.

---

11. Signals from the script

`Vm.NotifyHost(code)` hands a code to you, and does nothing if you are not listening:

```
static void OnNotify(int code, FlarisValue payload, void *userData)
{
    (void)userData;
    if (code == 7 && FlarisIsString(payload))
        SetStatusText(FlarisAsString(payload, NULL));
}

FlarisSetNotifyHandler(OnNotify, NULL);
```

```
fn Rebuild(items: [int]) {
    for (let i: int = 0; i < items.Length; i++) {
        Step(items[i]);
        VM.NotifyHost(7, "rebuilding " + str(i));
    }
}
```

The handler listens to the **calling thread's host**, and stays until that host is destroyed — `FlarisHostDestroy` removes it, so the next host on the thread starts without one. Pass `NULL` to stop listening sooner.

The payload is optional and may be any value; it is **borrowed**, so retain it if you keep it. The call returns `true` when a handler ran and `false` when none is installed.

That last part is the point of it. A script calling `Vm.NotifyHost` still compiles and runs under a host that ignores it, and under the plain `flarisvm` command — where it simply returns `false`. A native module cannot do that: a script calling `Host.Progress(...)` will not compile unless the module exists. Use a native module when you want types and a real name; use this when you want a channel that is always there.

It is cooperative. It fires only where the script chose to call it, so it is not a way to stop a script that never returns.

---

12. Errors and diagnostics

Script errors come back as values

A script that throws does not print and vanish. The call returns `FLARIS_ERR_RAISED` and writes the exception's code and message into your buffer:

```
char err[256];
int rc = FlarisFunctionCall(risky, NULL, 0, &result, err, sizeof err);

if (rc == FLARIS_ERR_RAISED)
    LogWarning("script error: %s", err);      /* e.g. "7: inventory full" */
```

`err` is always NUL-terminated and is cleared on entry, so a stale message never survives into a later call. The host stays usable afterwards — the next call runs normally.

Passing `NULL` (with size `0`) for the buffer is supported and skips formatting the message entirely. `rc` still tells you the call raised, so a host that only branches on failure and never shows the text can use it:

```
int rc = FlarisFunctionCall(risky, NULL, 0, &result, NULL, 0);  /* no message built */
```

That is worth doing only where raises are frequent enough to matter — building the message costs roughly a tenth of a microsecond per raised call. Where you report the error to a human or a log, take the buffer.

This holds for *any* failure inside the call, not just an explicit `throw`: a division by zero, an index out of bounds, or a blocked `Ffi` call with `unsafe` off all arrive the same way.

The load functions take no buffer: why a script failed to compile, or why its top level threw, is reported through the VM's diagnostics, below.

Nothing takes your process down

Nothing a script does, and nothing an input file contains, ends your process. An unhandled error is reported and unwound, a corrupt `.flx` is refused however deep the damage lies, a module whose pinned hash does not match is refused, a bundled dependency that fails the `requireSigned` check is withheld, and a module that fails to execute is refused — each as a return value, with the host usable afterwards. That is what

differs from the `flarisvm` command, which exits on all of these because exiting is the right thing for a command to do.

Two script-level constructs change meaning accordingly, because a script's decision to stop is not a decision that your application should stop:

In the script	Under `flarisvm`	Under your host
`Vm.Exit(code)`	exits with `code`	the call returns `FLARIS_ERR_RAISED`, naming the code
`Debug.AssertTrue` / `AssertEqual` failing	exits	raises, after printing the trace

Two things are outside that guarantee, and both are yours to control:

- **Allocator exhaustion is fatal only as a last resort.** When the object pool hits `opt.maxSlabs`, the allocation raises `Exception.OutOfMemory` out of a small reserve kept for exactly that, so the script (and your call) sees an ordinary raise, and it is told again if it exhausts the pool again. Only when the system itself refuses memory does the VM report and exit — there is no memory left to build an exception in. Size `maxSlabs` for your workload; the default ceiling is 1024 slabs.
- **The signal handlers are opt-out.** With `installSignals` left on, a `SIGINT`, `SIGTERM` or `SIGHUP` that no script handler has claimed ends the process immediately (exit code 128 + signal number) with nothing torn down. Set `opt.installSignals = FLARIS_OFF` to keep your own. The VM never handles `SIGSEGV`: a fault inside it reaches your handler, or the default. Destroying the last host that installed them puts the dispositions they replaced back, so a signal afterwards reaches whatever handler you had before — the VM's handler never runs against a destroyed host. `SIGPIPE` is the exception: the VM leaves it ignored, since a stray write to a closed peer should not kill your process either.

Capturing output

Everything a host writes for a reader goes through one handler: what a script prints with `Console` and `Debug`, and what the VM itself reports — compile errors and warnings, the "Errors detected" summary, a file that cannot be read, a top level that raised, an uncaught exception with its stack trace, a callback whose raise was dropped, and with `verbose` on, its progress chatter. Without a handler it all goes to stdout and stderr exactly as the `flarisvm` command writes it. Give the host one in its options instead:

```
static void MyOutput(int kind, const char *text, size_t len, void *userData)
{
    Request *req = userData;
    if (kind == FLARIS_OUTPUT)
        AppendToResponse(req, text, len);           /* what the script printed */
    else
        LogLine(req, kind, text);                   /* FLARIS_LOG_ERROR ... VERBOSE */
}

FlarisHostOptions opt = flarisHostOptionsDefaults;
opt.output         = MyOutput;
opt.outputUserData = req;
FlarisHostCreate(&opt, &host);
```

`kind` says what arrived:

- `FLARIS_OUTPUT` is script output, exactly as written: one `Console` or `Debug` call at a time, ending a line only where the script did — `Console.Write` arrives without a newline. A string can hold a zero byte, so trust `len`.
- `FLARIS_LOG_ERROR`, `FLARIS_LOG_WARNING`, `FLARIS_LOG_INFO` and `FLARIS_LOG_VERBOSE` are the VM's diagnostics, one whole message each, with no trailing newline. A report may span lines — an uncaught exception arrives with its stack trace in the same message — and a compile summary such as "Errors detected: 1" arrives as `FLARIS_LOG_INFO`, after the errors it counts.

`text` is NUL-terminated, carries no ANSI colour, and is valid only for the duration of the call, so copy it if you keep it. The handler belongs to the host: it receives everything from `FlarisHostCreate` — startup included — to the end of `FlarisHostDestroy`, teardown included.

With a handler there is no terminal to control, so `Console.SetColor`, the cursor calls and `Console.Clear` do nothing and `Console.GetCursor` returns `nil`. `Stream.Stdout` and `Stream.Stderr` are the process's own file descriptors and bypass the handler.

---

13. Threads: one host per thread

A host that wants parallelism starts more: **every thread may create a host of its own**, and any number of threads may do so at once. The code that runs one host runs one per thread unchanged:

```
static void *Worker(void *arg)
{
    FlarisHostOptions opt = flarisHostOptionsDefaults;
    opt.installSignals = FLARIS_OFF; /* let one thread own the process's signals */
    opt.ioThreads      = 2;          /* this host's own I/O workers */

    FlarisHost *host;
    if (FlarisHostCreate(&opt, &host) != FLARIS_OK)
        return NULL;

    if (FlarisHostLoadSource(host, (const char *)arg) == FLARIS_OK) {
        FlarisFunction *run = FlarisHostGetFunction(host, "Run");
        char err[256];
        if (run && FlarisFunctionCall(run, NULL, 0, NULL, err, sizeof err) != FLARIS_OK)
            fprintf(stderr, "%s: %s\n", (const char *)arg, err);
    }

    FlarisHostDestroy(host);
    return NULL;
}
```

What is isolated

Everything that makes a VM a VM. Two hosts share no objects, no globals, no classes, no fibers, no timers and no I/O workers, and nothing between them is locked on the calling path.

Native modules are per host too, which is the one thing that catches people out: give **each** thread's host its modules, in its options or with `FlarisRegisterModule` on that thread. Registering with one host publishes nothing to the others.

Values and functions do not travel either. A host and every `FlarisFunction` it handed out belong to the thread that created the host: used from another thread, each entry point returns `FLARIS_ERR_WRONG_THREAD` (`FlarisHostGetFunction` returns `NULL`) rather than corrupting either side. Passing a `FlarisValue` across is *not* detected — copy what you need over as plain C data.

`FlarisSetNotifyHandler` and the value constructors take no host argument: each acts on the calling thread and its host.

What is still shared, because the process has only one

Shared	What it means for you
Signal dispositions	The first host created with `installSignals` on installs them; they are restored when the last such host is destroyed. Leave it on for one thread, or off everywhere and handle signals yourself.
Compilation	Serialised across the process: two threads compiling at the same moment take turns. Compile once, or ship bytecode, rather than loading source in a hot loop on many threads.
Libraries loaded through `Ffi`	The mapping is shared and stays loaded while any host holds it, so the plugin's own code may run on several threads at once and its globals must be thread-safe. Callback registrations are *not* shared — a name each host registers resolves only for that host.
`FlarisSetArgs`	One argv for the process; the last caller wins for every host.
Environment and working directory	`Os.SetEnv` and `Os.Chdir` change them for the whole process: every host, and every child process started afterwards, sees the change. `Os` serialises its own environment reads and writes and its process launches across threads; a native module or FFI plugin that calls `getenv` / `setenv` itself is outside that lock. Child processes themselves are per host — `Os.KillChild` and the `Os` pipe calls accept only what that host's `Os.Spawn` returned.
`stdout` and `stderr`	Output from several hosts interleaves. Give each host an output handler (section 12) to keep each host's script output and diagnostics apart.
The process	A native module or an FFI plugin that crashes takes every host with it.

---

14. Embedding from C#

`libflaris` is a plain C shared library that exports 45 functions, all prefixed `Flaris`, and one constant, `flarisHostOptionsDefaults` — no other public surface. That makes it a direct P/Invoke target — no C++ name mangling, no wrapper layer, no generated glue.

Everything in this guide works from C#, including native modules whose functions are C# methods.

The pieces that need care

`FlarisValue` is one word. Pass it as `nuint`, or wrap it in a `readonly struct` over `nuint` so the compiler keeps handles apart from integers:

```
public readonly struct FlarisValue(nuint raw) { public readonly nuint Raw = raw; }
```

`FlarisHost *` and `FlarisFunction *` are opaque pointers. Hold them as `IntPtr`.

Use `[LibraryImport]`, not `[DllImport]`. It generates the marshalling at compile time, and `StringMarshalling.Utf8` gives you the UTF-8 the API expects:

```
[LibraryImport("flaris")]
public static partial int FlarisHostCreate(IntPtr options, out IntPtr host);

[LibraryImport("flaris")]
public static partial int FlarisHostDestroy(IntPtr host);

[LibraryImport("flaris", StringMarshalling = StringMarshalling.Utf8)]
public static partial int FlarisHostLoadScript(IntPtr host, string source, string name);

[LibraryImport("flaris", StringMarshalling = StringMarshalling.Utf8)]
public static partial IntPtr FlarisHostGetFunction(IntPtr host, string name);

[LibraryImport("flaris")]
public static partial int FlarisFunctionCall(IntPtr fn, nuint[]? args, int argc,
                                             out nuint result, byte[]? errBuf, nuint errSize);

[LibraryImport("flaris")]
public static partial nuint FlarisFunctionCallAsync(IntPtr fn, nuint[]? args, int argc);

[LibraryImport("flaris")]
public static partial int FlarisFiberStep(IntPtr host, nuint fiber, int maxWaitMs,
                                          out nuint result, byte[]? errBuf, nuint errSize);

[LibraryImport("flaris")]
public static partial int FlarisFiberCancel(IntPtr host, nuint fiber);
```

`IntPtr.Zero` for the options means defaults. To pass options, lay out `FlarisHostOptions` as a struct and pass a pointer to it.

Callbacks must be `[UnmanagedCallersOnly]`, not delegates. A delegate needs the GC to keep it alive and cannot be called from JIT-compiled Flaris code. An `[UnmanagedCallersOnly]` static method gives you a real function pointer:

```
[UnmanagedCallersOnly]
static FlarisValue DeviceRead(FlarisValue* args, int argc)
    => FlarisFloat(FlarisAsInt(args[0]) * 1.5);
```

The structs are sequential and blittable. `FlarisNative` holds a fixed-size array, so it needs `unsafe` and a `fixed` buffer:

```
[StructLayout(LayoutKind.Sequential)]
public unsafe struct FlarisNative
{
    public IntPtr Name;                                          // 0
```

```
    public delegate* unmanaged<FlarisValue*, int, FlarisValue> Fn;   // 8
    public uint ReturnType;                                          // 16
    public fixed uint ArgTypes[6];                                   // 20
    public byte Required, Optional;                                  // 44, 45
    public uint Flags;                                               // 48
}                                                                    // 56 bytes
```

`FlarisHostOptions` is 112 bytes and `FlarisModule` 24, and neither needs attributes beyond `LayoutKind.Sequential` — every field is an `int`, `uint`, `ushort`, `ulong` or pointer, in declaration order, and the natural alignment C# gives them is the C layout.

Registering a module of C# functions

```
var fns = stackalloc FlarisNative[1];
fns[0].Name       = Marshal.StringToHGlobalAnsi("Read");
fns[0].Fn         = &DeviceRead;
fns[0].ReturnType = T.Float;
fns[0].ArgTypes[0]= T.Int;
fns[0].Required   = 1;
fns[0].Flags      = FlarisNative.JitSafe;

FlarisHostCreate(IntPtr.Zero, out IntPtr host);
FlarisRegisterModule(host, "Device", fns, 1);
FlarisHostLoadScript(host, "fn Sample(p: int): float { return Device.Read(p); }", "s");
```

The script calls `Device.Read(p)` and your C# method runs. The VM copies the table and the names when you register, so `stackalloc` for the array is fine, and the unmanaged name may be freed once the call returns.

What is verified to work

A C# host built this way was checked against the library: creating and destroying hosts in a loop, loading a script, resolving a function once and calling it a million times with arguments and results crossing as `nuint`, releasing every value, and seeing a script exception arrive as a return code rather than an unwind through native frames.

The constraint that matters most

The threading rule in section 13 applies: **every call must come from the thread that created the host**. In C# that means no `async` continuation that might resume elsewhere, and no `Task.Run` around a call. If your application is `async`, marshal each host's work onto a single dedicated thread and keep it there.

Note also that a native module function runs on the host's thread inside the call that reached it: a C# method that blocks holds that call, and the thread, for as long as it waits.

---

15. Embedding from JavaScript

The VM also comes as a WebAssembly ES module that runs in a browser, a web worker and Node.js 18.3 or later. Download `flaris-lib-wasm.zip` from the downloads page - one archive serves every platform - and it unpacks to:

```
flaris-lib-wasm/
├── flaris.mjs              # the API below - the file you import
├── flaris-core.mjs         # the loader flaris.mjs imports
├── flaris-core.wasm        # the VM, compiler included
├── README.md
└── THIRD_PARTY_NOTICES.md
```

```
import { createFlaris, FlarisError } from "./flaris.mjs";

const vm = await createFlaris({ print: (line) => console.log(line) });
vm.load(`fn add(a, b) { return a + b; }`);

vm.call("add", 2, 3);          // 5
vm.call("add", "a", "b");      // "ab"

vm.shutdown();
```

It is the C API of sections 4-9 with the handles taken care of: each `createFlaris()` is one host, `load` runs a script into its one global scope, and `call` finds a function by name and calls it. Arguments and results are converted, so nothing needs releasing.

Call	Does
`createFlaris(options?)`	Start a host. `options.print` / `options.printErr` receive the script's stdout and stderr lines (default: `console.log` / `console.error`)
`vm.load(source, name?)`	Compile `source` and run its top level into the host's global scope, where a later load may redefine what an earlier one defined. `name` is what diagnostics call it (default `<host>`)
`vm.has(name)`	`true` when a script defined a callable `name`
`vm.call(name, ...args)`	Call it and return the converted result
`vm.callAsync(name, ...args, signal?)`	Start an `fn async` function and return a `Promise` of its converted result; the thread stays free while it waits - see below
`vm.registerModule(name, functions)`	Publish JavaScript functions a script calls as `name.fn(...)` - see below
`vm.fs`	The module's in-memory filesystem; write a `.flx` to `/libs` and a script can import it
`vm.shutdown()`	Destroy the host: its globals, fibers and timers. Nothing may be used afterwards

Values

JavaScript	Flaris	Back to JavaScript
`null`, `undefined`	`nil`	`null`
`true` / `false`	bool	boolean
a safe integer (`Number.isSafeInteger`)	int	number

JavaScript	Flaris	Back to JavaScript
any other number	float	number
`bigint` within 64 bits	int	number when safe, otherwise `bigint`
string	string (UTF-8)	string
array	array	array
plain object	object	object
—	char	a one-character string
—	class instance	a plain object of its fields
—	block	a `Uint8Array` copy
—	function, method, builtin	`undefined`; left out of an object

JavaScript has one number type, so an integral value such as `2.0` arrives as the int `2`. Values are copied each way: a script that changes an array it was given does not change the caller's array. Nesting is limited to 256 levels, which is also what stops a reference cycle. Anything else - a `Map`, a class instance, a typed array - throws a `TypeError`.

Callables never leave the VM, as in C: call a function by its name.

Calling JavaScript from a script

`registerModule` is section 9 for JavaScript. Each function becomes a member a script calls like a built-in namespace. Its arguments arrive converted the way a result is (the last column of the table above), and what it returns is converted the way an argument is:

```
vm.registerModule("Device", {
    read: (pin) => sensors[pin].value,
    status: () => ({ online: true, uptime: performance.now() }),
    log: { fn: (msg, level) => console.log(level ?? "info", msg), required: 1, optional: 1 },
});

vm.load(`fn Sample(pin) { return Device.read(pin) * 2.0; }`);   // after registering
```

- **Register before loading.** The compiler checks `Device.read(...)` against the registered module, so a script loaded first fails to compile. Names cannot be removed or replaced; taking a built-in namespace (`Math`, `Json`, ...) or an earlier module's name throws `registered`.
- **Arguments.** A plain function takes `fn.length` required arguments and up to six in all. `{ fn, required, optional }` sets the counts exactly, and a call with the wrong number is a compile error. Every argument and the result are typed `any`.
- **A throw is a script exception.** The script can catch it; `Code` is the error's integer `code` property when it has one, otherwise `6` (`Exception.RuntimeError`), and `Error` is its message. Uncaught, it reaches the JavaScript caller as a `FlarisError` of kind `raised`.
- **Synchronous only.** Returning a `Promise` raises in the script instead of waiting for it.
- **Calling back in is allowed.** A JavaScript function may `call` into the VM while the script that called it waits.

Calls that wait

`call` on an `fn async` function blocks the thread until the function returns - in a browser, the page freezes for as long as the script waits. `callAsync` runs it without blocking: it returns a `Promise` at once, the VM moves

the call on from the event loop, and the `Promise` settles when the function returns.

```
vm.load(`fn async Report(id) { Fiber.Sleep(150); return id * 2; }`);

const value = await vm.callAsync("Report", 21);                        // 42, 150 ms later
const quick = await vm.callAsync("Report", 1, AbortSignal.timeout(50)); // rejects after 50 ms
```

- **The page stays responsive.** While the script waits on a timer or sleep, nothing runs; the VM picks the call up again when that is due. A script that computes without waiting runs in slices, with the event loop free between them, so even a long computation does not freeze the page.
- **Cancel with an `AbortSignal`** as the last argument - `AbortSignal.timeout(ms)` for a deadline, an `AbortController` for a button. The call stops where it is, its `finally` blocks do not run, and the `Promise` rejects with the signal's reason, as `fetch` does.
- **Several at once.** Start as many as you like and `await` them together; they wait side by side, so four 30 ms sleeps take 30 ms. A VM runs at most 256 fibers at a time: a call past that rejects with `code` 18 (`Exception.OutOfFibers`).
- **Failures reject, never throw**: a script exception rejects with kind `raised`, a function that is not `fn async` with `arguments`, an unknown name with `not-found`, and a call still pending when the VM shuts down with `no-vm`.
- **`call` and `callAsync` mix.** A `call` made while async calls are pending runs at once; those calls go on afterwards.

Errors

Every failure throws a `FlarisError`, whose `kind` says what happened:

`kind`	When
`raised`	The script threw. `code` is its exception code and `message` its message
`compile`	`load` failed to compile. `message` holds the diagnostics; warnings of a successful load go to `printErr` instead
`suspended`	A plain function or a top level slept, yielded or awaited - only an `fn async` function can wait. From `callAsync`: nothing left could ever wake the call
`not-found`	`call` named no function a script defined
`arguments`	An argument nested deeper than 256 levels, or contained a cycle
`registered`	`registerModule` was given a name already taken
`collision`	Two names in a `registerModule` table share a hash; rename one
`init`	`createFlaris` could not start the VM
`busy`	`shutdown` was called from a JavaScript function a script is running; shut down after the call returns
`no-vm`	The host was shut down

```
try { vm.call("charge", order); }
catch (e) {
    if (e instanceof FlarisError && e.kind === "raised") console.log(e.code, e.message);
    else throw e;
}
```

Serving it

- **Keep the three files together.** `flaris.mjs` imports the loader, and the loader fetches the `.wasm`, each relative to its own URL - so the directory can live anywhere, but not be split up.

- **Serve over HTTP(S).** Browsers do not import modules into a page opened from disk.
- **Serve `.wasm` as `application/wasm`.** Most servers do. One that does not still works - the browser compiles from a buffer instead of while downloading, which only makes start-up slower - but on Apache `AddType application/wasm .wasm` fixes it.
- **From another origin** (a CDN, a separate static host), the server must send `Access-Control-Allow-Origin` for all three files, or the browser refuses the import.
- **Compress it.** The `.wasm` is about 860 KB raw and 300 KB with brotli, so turn on compression for it if your host does not already.
- **In Node**, import `flaris.mjs` by its path; there is nothing to serve.

What it does not do yet

- **A host function cannot return a `Promise`.** A script cannot wait for JavaScript work such as `fetch`; the function must return its value.
- **No JIT.** The WebAssembly VM interprets, at roughly 4x the native interpreter's time.
- **One thread.** `call` and `load` block the thread they run on, so run long synchronous work in a web worker. A runaway loop inside `call` cannot be interrupted from the same thread; terminate the worker. Inside `callAsync` it runs in slices and an `AbortSignal` stops it.
- **No processes, sockets, FFI or TLS** - the limits of the browser sandbox.

---

16. Limits and what is not supported

One host per thread. A thread holds at most one host at a time, and every script loaded into it shares its one global scope. Scripts that must not see each other need separate hosts, which means separate threads (section 13). Neither a function nor a value crosses from one host to another.

Only `fn async` functions can wait, and nothing runs between calls. A call to an `fn async` function blocks until it returns, running timers, I/O and other fibers meanwhile; a plain function or a load that waits returns `FLARIS_ERR_SUSPENDED` and is abandoned, and work a call leaves behind moves on only while a later call waits or a fiber is stepped (section 7). For parallelism, start a second host on a second thread.

A running call cannot be stopped. There is currently no way to interrupt a call from another thread, and the VM has no timeout of its own: a script that loops forever holds the calling thread for good. A wait can be bounded — start the call with `FlarisFunctionCallAsync` and step it with your own deadline (section 7) — but a loop inside a function compiled to native code still runs to its end before the step returns. Recursion depth (`maxFrames`) and memory (`maxSlabs`) are bounded per host; time is not. Code you do not trust at all belongs in a process you can kill.

A native module is bound to the host that registers it. Section 9 covers this in full: a `.flx` containing calls into your module runs only in a host that registers those exact names, and the standalone `flarisvm --compile` cannot compile a script that calls one, because it does not know your module. Compile such scripts from your own host, at load.

A malformed `.flx` is refused, not survived halfway. The bytecode reader abandons the whole load on the first inconsistency — `FlarisHostLoadProgram` returns `FLARIS_ERR_COMPILE` and nothing from that file is defined. It does not attempt partial recovery, so a damaged file is never half-loaded.

An FFI plugin remains the alternative when the code you want to reach lives in a shared library you do not compile against: the script loads and calls it through `Ffi`, which requires `FLARIS_CAP_FFI` (or `FLARIS_CAP_UNSAFE`, which implies it) and is refused outright when `FLARIS_MOD_FFI` is denied. A native module needs neither.

---

17. API reference

Everything below is declared in `flaris.h`.

Hosts

Function	Description
`int FlarisHostCreate(const FlarisHostOptions *options, FlarisHost **outHost)`	Start a host on this thread. `NULL` options means defaults. `FLARIS_ERR_INIT` when an option is rejected, `FLARIS_ERR_BUSY` when the thread already has a host, `FLARIS_ERR_ARGS` / `REGISTERED` / `COLLISION` for a bad module table. Nothing is left running on failure.
`int FlarisHostDestroy(FlarisHost *host)`	Tear it down; every value and function handle it gave out dies. `FLARIS_ERR_BUSY` from inside one of its own calls, `FLARIS_ERR_WRONG_THREAD` from another thread.
`const FlarisHostOptions flarisHostOptionsDefaults`	Every field `FLARIS_DEFAULT` / 0: the behaviour the `flarisvm` command runs with.
`FlarisHostOptions.output` / `.outputUserData`	Where script output (`FLARIS_OUTPUT`) and diagnostics (`FLARIS_LOG_*`) go; `NULL` is stdout and stderr as the `flarisvm` command writes them.
`void FlarisSetArgs(int argc, char **argv)`	Make argv visible to `Os.Args`. Borrows `argv`; one for the whole process.

Loading

Function	Description
`int FlarisHostLoadProgram(host, const char *path)`	Load a `.flx` and run its top level.
`int FlarisHostLoadSource(host, const char *path)`	The same, compiling a `.fls`. Not in a runtime-only build.
`int FlarisHostLoadScript(host, const char *source, const char *name)`	The same, from memory. `name` labels diagnostics; `NULL` means `"<host>"`. Not in a runtime-only build.

All three land in the host's one global scope and return `FLARIS_OK`, `FLARIS_ERR_COMPILE`, `FLARIS_ERR_RAISED` or `FLARIS_ERR_SUSPENDED`.

Calling

Function	Description
`FlarisFunction *FlarisHostGetFunction(host, const char *name)`	The callable `name` as defined now, or `NULL`. Owned by the host until it is destroyed; nothing to release. The same function always gives the same handle.
`int FlarisFunctionCall(fn, args, argc, outResult, errBuf, errSize)`	Call it. Borrows `args`; the result is owned (`NULL` releases it for you). `errBuf` receives `"code: message"` on a raise and is cleared on entry.
`FlarisValue FlarisFunctionCallAsync(fn, args, argc)`	Start an `fn async` function: its fiber, owned, queued but not run. Nil when the call itself is wrong.

Function	Description
`int FlarisFiberStep(host, fiber, maxWaitMs, outResult, errBuf, errSize)`	Move the host on until the fiber finishes or `maxWaitMs` passes (`0` one pass, `-1` no limit). `FLARIS_PENDING` until then; afterwards the same answer every time.
`int FlarisFiberCancel(host, fiber)`	Stop the fiber where it is; its steps then answer `FLARIS_ERR_SUSPENDED`.

Signals from the script

Function	Description
`void FlarisSetNotifyHandler(FlarisNotifyFn fn, void *userData)`	Listen for `Vm.NotifyHost(code[, payload])` from this thread's host, until it is destroyed. `NULL` stops listening.

Sandboxing

`FlarisHostOptions.deniedModules` (`FLARIS_MOD_*`, 0 allows everything) and `.grantCaps` / `.denyCaps` (`FLARIS_CAP_*`, deny applied after grant) — sections 5 and 10.

Native modules

Function	Description
`FlarisHostOptions.modules` / `.moduleCount`	An array of `FlarisModule { name, fns, count }`, registered at create.
`int FlarisRegisterModule(host, const char *name, const FlarisNative *fns, int count)`	Register one after create. Only scripts loaded afterwards can call it.
`FLARIS_FN_JIT_SAFE`	Flag: native code may call this function directly; see the promise in section 9.
`FLARIS_FN_PURE`	Flag: no observable effect, result depends only on the arguments. Implies JIT-safe.

Both forms copy the table and its names.

Values

`FlarisValue` is an opaque handle to a value inside the VM. Constructors return an owned value; readers return borrowed data valid while the value is.

Function	Description
`FlarisNil()` / `FlarisBool(v)` / `FlarisInt(v)` / `FlarisFloat(v)`	Scalars.
`FlarisString(s)` / `FlarisStringN(s, len)`	Strings.
`FlarisIsNil(v)` / `FlarisIsBool` / `FlarisIsInt` / `FlarisIsFloat`	Scalar type tests.
`FlarisIsString(v)` / `FlarisIsArray(v)` / `FlarisIsObject(v)`	Container type tests.
`FlarisIsInstance(v)` / `FlarisClassName(v)`	A class instance, and its class name.
`FlarisIsCallable(v)`	A function, bound method, builtin or FFI function.
`FlarisAsInt(v)` / `FlarisAsFloat(v)` / `FlarisAsBool(v)`	Numeric reads; 0 on a type mismatch.
`const char *FlarisAsString(v, &len)`	Borrowed chars, `NULL` for a non-string.
`void *FlarisBlockData(v, &len)`	A Block's bytes directly — bulk data with no per-element call.
`FlarisArrayNew(cap)` / `FlarisArrayLen(a)` / `FlarisArrayAppend(a, v)`	Arrays; append consumes `v`.

Function	Description
`FlarisArrayGet(a, i)`	**Owned** element — release it.
`FlarisArrayGetInt(a, i)` / `FlarisArrayGetFloat(a, i)`	Numeric elements without boxing.
`FlarisObjectNew()` / `FlarisObjectLen(o)` / `FlarisObjectGet(o, key)`	Objects; `Get` borrows.
`FlarisObjectSet(o, key, v)`	Consumes `v`. Works on instances too.
`FlarisObjectNext(o, &cursor, &key, &keyLen, &value)`	Walk every property in insertion order. Both borrowed.
`FlarisRetain(v)` / `FlarisReleaseValue(v)`	Keep a borrowed value / drop an owned one.

Status codes

Constant	Value	Meaning
`FLARIS_OK`	0	Success
`FLARIS_PENDING`	1	A stepped fiber has not finished yet — not a failure
`FLARIS_ERR_ARGS`	-3	A bad argument: a `NULL` handle, `args` `NULL` with `argc` > 0, `argc` outside 0–16, or a malformed module table
`FLARIS_ERR_RAISED`	-4	The script threw
`FLARIS_ERR_SUSPENDED`	-5	It waited where it cannot - a plain function, a load, or a nested call - and was abandoned; or a stepped fiber was cancelled
`FLARIS_ERR_INIT`	-10	An option was rejected, or the host could not start
`FLARIS_ERR_COMPILE`	-11	The script did not compile, or the file did not load
`FLARIS_ERR_REGISTERED`	-13	That module name is already taken
`FLARIS_ERR_COLLISION`	-14	Two names share a 32-bit hash; rename one
`FLARIS_ERR_BUSY`	-15	This thread already has a host; or you are inside a native module function, where the host is running below you and cannot be destroyed, stepped or have a fiber cancelled
`FLARIS_ERR_WRONG_THREAD`	-17	The host belongs to another thread (section 13)